САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ

ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

Отчет по лабораторной работе №3

по дисциплине «Методы оптимизации»

Тема: «Решение задач многомерной условной оптимизации методом множителей Лагранжа»

Вариант: 2

Выполнил: Бен Шамех Абделазиз
Группа: К3339
Поток: МетОпт 1.2
Преподаватель: Лаврентьев Савелий

Санкт-Петербург
2026 г.

1. Цель работы

Освоить метод множителей Лагранжа для поиска условных экстремумов функций нескольких переменных при ограничениях-равенствах: научиться проверять применимость метода, находить все точки-кандидаты, классифицировать их и отдельно обосновывать, являются ли найденные экстремумы глобальными. Решение выполняется вручную и программно (Python, SymPy), после чего результаты сопоставляются.

2. Номер варианта и постановка задачи

Вариант 2. Найти глобальные минимум и максимум функции

$$f(x, y) = 2x - y \quad \text{при ограничении} \quad x^2 + y^2 = 5.$$

Переменные: $x, y \in \mathbb{R}$, $n = 2$. Ограничение приводится к виду $h(x, y) = 0$:

$$h(x, y) = x^2 + y^2 - 5 = 0, \qquad m = 1.$$

3. Анализ применимости метода

1. Функции $f(x, y) = 2x - y$ (линейная) и $h(x, y) = x^2 + y^2 - 5$ (многочлен) определены на всей плоскости $\mathbb{R}^2$ и имеют непрерывные производные любого порядка. Значит, гладкость, нужная для метода (и для второго порядка), обеспечена.

2. Допустимое множество $D = \{(x, y) : x^2 + y^2 = 5\}$ — окружность радиуса $\sqrt{5}$ с центром в начале координат. Оно непусто: например, точка $(\sqrt{5}, 0)$ или $(2, 1)$, так как $4 + 1 = 5$.

3. Градиенты:
$$\nabla f = (2,\, -1), \qquad \nabla h = (2x,\, 2y).$$

4. Регулярность. Матрица Якоби ограничения $J_h = (2x \;\; 2y)$. Она имеет ранг меньше $m = 1$ только если $x = y = 0$. Но $h(0, 0) = -5 \neq 0$, то есть начало координат не лежит на D. Следовательно, в каждой допустимой

точке $\nabla h \neq 0$, ранг J_h равен $1 = m$, и все точки D регулярны. Особых точек нет.

5. Число ограничений $m = 1 < n = 2$, остается одна степень свободы (движение по окружности).

Вывод: f и h непрерывно дифференцируемы, $D \neq \emptyset$, градиент ограничения нигде на D не обращается в нуль, поэтому метод множителей Лагранжа применим и дает необходимые условия экстремума во всех допустимых точках. Глобальность придется обосновывать отдельно.

4. Существование глобальных экстремумов

Множество D замкнуто (задано равенством для непрерывной функции) и ограничено ($|x| \leq \sqrt{5}$, $|y| \leq \sqrt{5}$), то есть компактно. Функция f непрерывна. По теореме Вейерштрасса f достигает на D и наименьшего, и наибольшего значения, так что глобальные минимум и максимум существуют.

5. Функция Лагранжа

Используем соглашение $L = f + \lambda h$:

$$L(x, y, \lambda) = 2x - y + \lambda\,(x^2 + y^2 - 5).$$

6. Система стационарности

$$\begin{cases} \dfrac{\partial L}{\partial x} = 2 + 2\lambda x = 0, \\ \dfrac{\partial L}{\partial y} = -1 + 2\lambda y = 0, \\ x^2 + y^2 - 5 = 0. \end{cases}$$

Это три уравнения с тремя неизвестными x, y, λ ($n + m = 3$).

7. Ручное решение

Если $\lambda = 0$, первое уравнение дает $2 = 0$, что невозможно. Значит $\lambda \neq 0$, и из первых двух уравнений

$$x = -\frac{1}{\lambda}, \qquad y = \frac{1}{2\lambda}.$$

Подставляем в ограничение:

$$\frac{1}{\lambda^2} + \frac{1}{4\lambda^2} = 5 \implies \frac{5}{4\lambda^2} = 5 \implies \lambda^2 = \frac{1}{4} \implies \lambda = \pm\frac{1}{2}.$$

- $\lambda = \frac{1}{2}$: $x = -2,\ y = 1.$
- $\lambda = -\frac{1}{2}$: $x = 2,\ y = -1.$

Других действительных решений у системы нет, так как все ее решения получены однозначным выражением через λ, а для λ получено квадратное уравнение.

8. Таблица кандидатов

№	Точка (x, y)	λ	Невязка h	$f(x, y)$	Классификация
1	$(2,\ -1)$	$-\frac{1}{2}$	0	5	строгий локальный максимум
2	$(-2,\ 1)$	$\frac{1}{2}$	0	-5	строгий локальный минимум

9. Проверка ограничений

$$h(2, -1) = 4 + 1 - 5 = 0, \qquad h(-2, 1) = 4 + 1 - 5 = 0.$$

Обе точки допустимы. Значения целевой функции:

$$f(2, -1) = 4 + 1 = 5, \qquad f(-2, 1) = -4 - 1 = -5.$$

10. Классификация кандидатов

Гессиан функции Лагранжа по (x, y):

$$\nabla^2_{xx} L = \begin{pmatrix} 2\lambda & 0 \\ 0 & 2\lambda \end{pmatrix}.$$

Касательное направление d определяется условием $J_h d = 0$.

Точка $(-2, 1)$, $\lambda = \frac{1}{2}$. $\nabla h = (-4,\ 2)$, откуда $d = (1,\ 2)$. Тогда

$$d^T \nabla^2_{xx} L\, d = 2 \cdot \tfrac{1}{2}\,(1^2 + 2^2) = 5 > 0,$$

квадратичная форма на касательном направлении положительна — строгий локальный минимум.

Точка $(2, -1)$, $\lambda = -\frac{1}{2}$. $\nabla h = (4,\ -2)$, откуда снова $d = (1,\ 2)$. Тогда

$$d^T \nabla^2_{xx} L\, d = 2 \cdot \left(-\tfrac{1}{2}\right)(1 + 4) = -5 < 0,$$

форма отрицательна — строгий локальный максимум.

Независимая проверка через параметризацию

Окружность параметризуется как $x = \sqrt{5}\cos t$, $y = \sqrt{5}\sin t$. Тогда

$$f(t) = \sqrt{5}\,(2\cos t - \sin t) = 5\cos(t + \varphi), \qquad \cos\varphi = \tfrac{2}{\sqrt{5}},\ \sin\varphi = \tfrac{1}{\sqrt{5}}.$$

Значит, $-5 \le f \le 5$. Максимум 5 достигается при $t = -\varphi$, то есть в точке $(\sqrt{5}\cos\varphi,\ -\sqrt{5}\sin\varphi) = (2, -1)$, минимум -5 — при $t = \pi - \varphi$, то есть в точке $(-2, 1)$. Это совпадает с результатом метода Лагранжа.

11. Обоснование глобальности

Глобальность следует не из самой системы Лагранжа, а из следующей цепочки:

1. D компактно, f непрерывна, значит глобальные минимум и максимум существуют (теорема Вейерштрасса);

2. все точки D регулярны, поэтому любой экстремум обязан быть решением системы Лагранжа;

3. система имеет ровно два действительных решения, список кандидатов полон;

4. $f(-2,1) = -5 < f(2,-1) = 5$.

Следовательно, $(-2,1)$ — точка глобального минимума ($f_{\min} = -5$), $(2,-1)$ — точка глобального максимума ($f_{\max} = 5$).

Дополнительно оценка подтверждается неравенством Коши–Буняковского: для любой допустимой точки

$$|2x - y| \le \sqrt{2^2 + (-1)^2}\,\sqrt{x^2 + y^2} = \sqrt{5} \cdot \sqrt{5} = 5,$$

причем равенство достигается ровно в найденных точках.

12. Программное решение

12.1 Описание программы

Программа написана на Python 3 с использованием SymPy. Готовые оптимизаторы (`scipy.optimize` и подобные) не применяются: программа шаг за шагом воспроизводит ручное решение. Ниже показаны ключевые фрагменты кода в том порядке, в каком они выполняются; полный листинг приведен в приложении, а исходный файл лежит в репозитории: `https://github.com/AbdelazizBenChamekh/Optimization_Methods/tree/main/Lab3`.

Задание функции и ограничения

Целевая функция и ограничение записаны прямо в коде, ограничение сразу в виде $h(x) = 0$. Допуск `EPS` вынесен в отдельную константу. Флаги компактности и полноты списка кандидатов выставлены вручную по результатам анализа задачи: программа не умеет доказывать их сама.

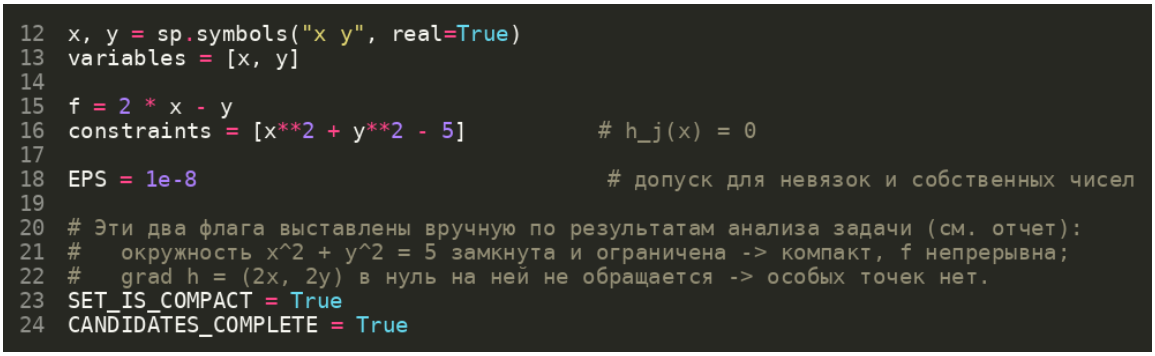


Рисунок 1 — Задание варианта 2 и настройки программы

Функция Лагранжа и система

Множители создаются автоматически по числу ограничений, лагранжиан строится как $L = f + \sum \lambda_j h_j$. Система состоит из производных $\partial L / \partial x_i$ и исходных ограничений ($n + m$ уравнений).

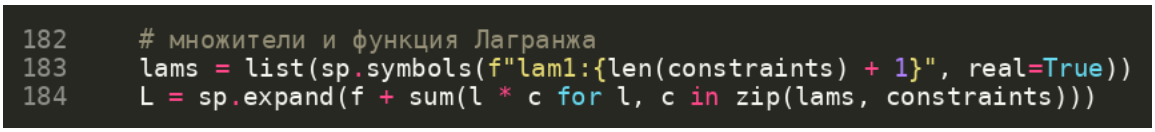


Рисунок 2 — Построение функции Лагранжа

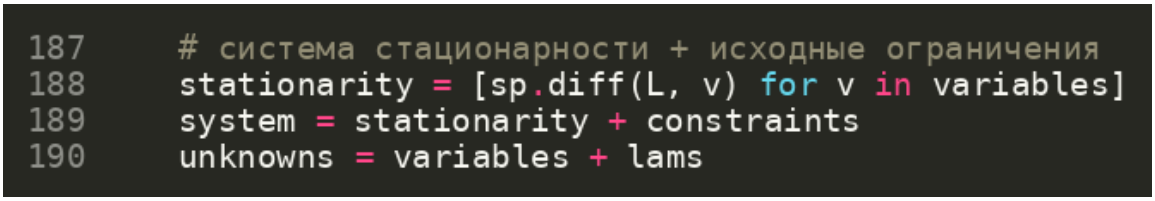


Рисунок 3 — Формирование системы уравнений

Поиск всех действительных решений

Функция `solve_all` решает систему двумя решателями SymPy, `solve` и `nonlinsolve`, и объединяет результаты. Комплексные корни и дубликаты отбрасываются, параметрические семейства помечаются отдельно, так как точками-кандидатами они не являются.

```python
def solve_all(equations, unknowns):
    """Решаем систему двумя решателями sympy и объединяем результаты."""
    points, family, used = [], False, []

    try:
        raw = sp.solve(equations, unknowns, dict=True)
        pts, fam = parse_solutions(raw, unknowns)
        points += pts
        family |= fam
        used.append("solve")
    except Exception as err:
        print("  solve не отработал:", err)

    try:
        raw = sp.nonlinsolve(equations, unknowns)
        pts, fam = parse_solutions(raw, unknowns)
        for p in pts:
            if not any(same_point(p, q, unknowns) for q in points):
                points.append(p)
        family |= fam
        used.append("nonlinsolve")
    except Exception as err:
        print("  nonlinsolve не отработал:", err)

    if points:
        status = "конечное число решений"
    elif family:
        status = "параметрическое семейство"
    elif used:
        status = "решений нет"
    else:
        status = "решить не удалось"
    return {"points": points, "family": family, "status": status, "methods": used}
```

Рисунок 4 — Функция solve_all

Особые точки

Функции `jacobian` и `singular_system` строят матрицу Якоби и систему для поиска точек, в которых ранг Якоби падает (ограничение плюс равенство нулю всех миноров порядка m). Для варианта 2 такая система решений не имеет.

```python
def jacobian():
    return sp.Matrix([[sp.diff(c, v) for v in variables] for c in constraints])


def singular_system():
    """h_j = 0 и равенство нулю всех миноров Якоби порядка m (там ранг < m)."""
    J = jacobian()
    m, n = J.shape
    eqs = list(constraints)
    if m <= n:
        for cols in combinations(range(n), m):
            eqs.append(sp.expand(J[:, list(cols)].det()))
    return eqs
```

Рисунок 5 — Матрица Якоби и поиск особых точек

Отбор кандидатов и проверка ограничений

Каждое найденное решение подставляется в исходное ограничение, и точка принимается только при нулевой невязке. Особые точки, если они есть,

добавляются к списку отдельно.

```
    # собираем кандидатов
    cands = []
    for sol in res["points"]:
        pt = {v: sol[v] for v in variables}
        if not all(is_zero(c.subs(pt)) for c in constraints):
            print("  отброшено, ограничение не выполнено:", show_point(pt, variables))
            continue
        cands.append({"pt": pt, "lam": {l: sol[l] for l in lams}, "kind": "Лагранж"})
    for sol in sing["points"]:
        pt = {v: sol[v] for v in variables}
        if any(same_point(pt, c["pt"], variables) for c in cands):
            continue
        cands.append({"pt": pt, "lam": None, "kind": "особая"})
```

Рисунок 6 — Отбор кандидатов

Анализ кандидатов

Для каждой точки вычисляются значение f, невязка, допустимость, ранг матрицы Якоби и классификация второго порядка.

```
    # анализ каждого кандидата
    J = jacobian()
    m = len(constraints)
    print("\nКандидаты:")
    for i, c in enumerate(cands, 1):
        pt = c["pt"]
        c["value"] = sp.simplify(f.subs(pt))
        c["resid"] = [sp.simplify(h.subs(pt)) for h in constraints]
        c["feasible"] = all(is_zero(r) for r in c["resid"])
        c["rank"] = int(J.subs(pt).rank())
        if c["lam"] is not None:
            c["class"] = classify(L, {**pt, **c["lam"]})
        else:
            c["class"] = "особая точка, метод Лагранжа неприменим"

        print(f"\nКандидат {i} ({c['kind']})")
        print("  точка:", show_point(pt, variables))
        if c["lam"] is not None:
            print("  множители:", show_point(c["lam"], lams))
        print("  f =", c["value"])
        print("  невязки:", c["resid"])
        print("  допустимость:", "да" if c["feasible"] else "нет")
        print(f"  ранг Якоби: {c['rank']} из {m}")
        print("  классификация:", c["class"])
```

Рисунок 7 — Вычисление характеристик кандидата и их вывод

Классификация второго порядка

Функция `classify` находит базис касательного подпространства (ядро J_h), сужает гессиан L на него и по знакам собственных чисел определяет тип точки.

```python
def classify(L, values):
    """Второй порядок: гессиан L сужаем на касательное подпространство."""
    J = jacobian().subs(values)
    basis = J.nullspace()
    if not basis:
        return "касательное пространство нулевое, второй порядок не применим"
    B = sp.Matrix.hstack(*basis)
    H = sp.hessian(L, variables).subs(values)
    Hr = sp.simplify(B.T * H * B)

    eig = []
    for val, mult in Hr.eigenvals().items():
        num = complex(sp.N(val))
        if abs(num.imag) > EPS:
            return "комплексные собственные числа, классификация не удалась"
        eig += [num.real] * mult

    if all(e > EPS for e in eig):
        return "строгий локальный минимум"
    if all(e < -EPS for e in eig):
        return "строгий локальный максимум"
    if any(e > EPS for e in eig) and any(e < -EPS for e in eig):
        return "не экстремум (седло)"
    return "второй порядок ответа не дает"
```

Рисунок 8 — Функция classify

Выбор минимума и максимума

Глобальные экстремумы объявляются только если одновременно выполнены все условия: множество компактно, список кандидатов полон, система Лагранжа дала конечное число решений без семейств, поиск особых точек завершился корректно, все кандидаты допустимы. Иначе программа сообщает, что нужен ручной анализ.

```python
    can_declare = (
        SET_IS_COMPACT and CANDIDATES_COMPLETE and cands
        and res["points"] and not res["family"]
        and sing["status"] in ("решений нет", "конечное число решений")
        and not sing["family"]
        and all(c["feasible"] for c in cands)
    )
    if not can_declare:
        print("Глобальные экстремумы автоматически не объявлены:")
        print("не выполнено хотя бы одно из условий (компактность, полнота")
        print("списка, конечность решений, допустимость). Нужен ручной анализ.")
        return

    best = min(cands, key=lambda c: float(sp.N(c["value"])))
    worst = max(cands, key=lambda c: float(sp.N(c["value"])))
    print("Глобальный минимум:", show_point(best["pt"], variables), "f =", best["value"])
    print("Глобальный максимум:", show_point(worst["pt"], variables), "f =", worst["value"])
    print("Основание: допустимое множество компактно, f непрерывна,")
    print("кандидаты перечислены полностью, значения f сравнены.")
```

Рисунок 9 — Условия объявления глобальных экстремумов

12.2 Результаты запуска

Запуск программы для варианта 2:

```
Целевая функция: f = 2*x - y
Ограничения:
  h1 = x**2 + y**2 - 5 = 0

Функция Лагранжа:
  L = lam1*x**2 + lam1*y**2 - 5*lam1 + 2*x - y

Уравнения стационарности:
  dL/dx = 2*lam1*x + 2 = 0
  dL/dy = 2*lam1*y - 1 = 0
Ограничения:
  h1 = x**2 + y**2 - 5 = 0

Матрица Якоби ограничений: [2*x, 2*y]

Решение системы Лагранжа:
  решатели: solve, nonlinsolve
  статус: конечное число решений
  найдено действительных решений: 2

Поиск особых точек (rank J < m):
  статус: решений нет
  найдено точек: 0
```

Рисунок 10 — Построение системы Лагранжа и ее решение

```
Кандидаты:

Кандидат 1 (Лагранж)
  точка: (x = 2, y = -1)
  множители: (lam1 = -1/2)
  f = 5
  невязки: [0]
  допустимость: да
  ранг Якоби: 1 из 1
  классификация: строгий локальный максимум

Кандидат 2 (Лагранж)
  точка: (x = -2, y = 1)
  множители: (lam1 = 1/2)
  f = -5
  невязки: [0]
  допустимость: да
  ранг Якоби: 1 из 1
  классификация: строгий локальный минимум

==================================================
Итог
==================================================
Глобальный минимум: (x = -2, y = 1) f = -5
Глобальный максимум: (x = 2, y = -1) f = 5
Основание: допустимое множество компактно, f непрерывна,
кандидаты перечислены полностью, значения f сравнены.
```

Рисунок 11 — Кандидаты и итог

13. Сопоставление ручного и программного решений

Характеристика	Ручное решение	Программное решение
Точки-кандидаты	$(2,-1)$ и $(-2,1)$	$(2,-1)$ и $(-2,1)$
Множители	$\lambda=-\frac{1}{2}$ и $\lambda=\frac{1}{2}$	lam1 = $-1/2$ и lam1 = $1/2$
Значения функции	5 и -5	5 и -5
Минимум	$f(-2,1)=-5$	$f(-2,1)=-5$
Максимум	$f(2,-1)=5$	$f(2,-1)=5$
Классификация	макс. в $(2,-1)$, мин. в $(-2,1)$	то же

Расхождений нет. Результаты ручного решения, программы и независимой проверки через параметризацию окружности совпали.

14. Вывод

Применимость метода. Метод Лагранжа применим: f и h гладкие, окружность $x^2+y^2=5$ непуста, а $\nabla h=(2x,2y)$ не обращается в нуль ни в одной ее точке, поэтому ранг матрицы Якоби равен $1=m$ везде на D.

Найденные точки. Система Лагранжа дает две точки: $(2,-1)$ при $\lambda=-\frac{1}{2}$ и $(-2,1)$ при $\lambda=\frac{1}{2}$. Обе допустимы, невязки равны нулю.

Характер экстремумов. По знаку квадратичной формы на касательном направлении $(2,-1)$ — строгий локальный максимум, $(-2,1)$ — строгий локальный минимум.

Глобальность. Экстремумы глобальные: окружность компактна, f непрерывна, регулярность выполнена во всех точках, список кандидатов полон, а сравнение значений дает $f_{\min}=-5$ в точке $(-2,1)$ и $f_{\max}=5$ в точке $(2,-1)$. Результат дополнительно подтвержден параметризацией и неравенством Коши–Буняковского.

Совпадение решений. Ручное и программное решения полностью совпали.

Ограничения программного подхода. Программа работает только с ограничениями-равенствами; для неравенств нужны условия ККТ. Символьные решатели SymPy не всегда находят все корни, а при параметрическом семействе программа не объявляет результат полным. Компактность множества и полнота списка кандидатов задаются вручную. Проверки невязок и собственных чисел используют численный допуск 10^{-8}.

Приложение. Листинг программы lab3.py

```
"""
Лабораторная работа №3 по МетОпт.
Метод множителей Лагранжа, вариант 2:
    f(x, y) = 2x - y  ->  extr,   x^2 + y^2 = 5
"""
from itertools import combinations

import sympy as sp

# ---------- условие задачи ----------

x, y = sp.symbols("x y", real=True)
variables = [x, y]

f = 2 * x - y
constraints = [x**2 + y**2 - 5]          # h_j(x) = 0

EPS = 1e-8                               # допуск для невязок и собственных чисел

# Эти два флага выставлены вручную по результатам анализа задачи (см. отчет):
#   окружность x^2 + y^2 = 5 замкнута и ограничена -> компакт, f непрерывна;
#   grad h = (2x, 2y) в нуль на ней не обращается -> особых точек нет.
SET_IS_COMPACT = True
CANDIDATES_COMPLETE = True


# ---------- вспомогательные функции ----------

def is_real_number(v):
    """Значение должно быть конкретным действительным числом."""
    v = sp.simplify(v)
    if v.free_symbols:
        return False
    if v.is_real is not None:
        return bool(v.is_real) and v.is_finite is not False
    try:
        return abs(complex(sp.N(v)).imag) <= EPS
    except (TypeError, ValueError):
        return False


def is_zero(expr):
    """Равно ли выражение нулю (точно или с допуском EPS)."""
    expr = sp.simplify(expr)
    if expr == 0:
        return True
    try:
        return abs(complex(sp.N(expr))) <= EPS
    except (TypeError, ValueError):
        return False


def same_point(a, b, names):
    return all(sp.simplify(a[n] - b[n]) == 0 for n in names)


def parse_solutions(raw, unknowns):
    """
```

```
    Приводит ответ solve / nonlinsolve к списку словарей {символ: значение}.
    Возвращает (список точек, флаг "решатель вернул семейство").
    """
    found, family = [], False
    if raw is None:
        return found, True
    if isinstance(raw, (dict, sp.Tuple, tuple)):
        raw = [raw]
    try:
        items = list(raw)
    except TypeError:
        return found, True

    for item in items:
        if isinstance(item, dict):
            sol = dict(item)
        elif len(item) == len(unknowns):
            sol = dict(zip(unknowns, item))
        else:
            family = True
            continue
        if any(u not in sol for u in unknowns):
            family = True                      # какая-то переменная свободна
            continue
        sol = {u: sp.simplify(sol[u]) for u in unknowns}
        if any(val.free_symbols for val in sol.values()):
            family = True                      # в ответе остался параметр
            continue
        if not all(is_real_number(val) for val in sol.values()):
            continue                           # комплексные корни не берем
        if not any(same_point(sol, old, unknowns) for old in found):
            found.append(sol)
    return found, family


def solve_all(equations, unknowns):
    """Решаем систему двумя решателями sympy и объединяем результаты."""
    points, family, used = [], False, []

    try:
        raw = sp.solve(equations, unknowns, dict=True)
        pts, fam = parse_solutions(raw, unknowns)
        points += pts
        family |= fam
        used.append("solve")
    except Exception as err:
        print("  solve не отработал:", err)

    try:
        raw = sp.nonlinsolve(equations, unknowns)
        pts, fam = parse_solutions(raw, unknowns)
        for p in pts:
            if not any(same_point(p, q, unknowns) for q in points):
                points.append(p)
        family |= fam
        used.append("nonlinsolve")
    except Exception as err:
        print("  nonlinsolve не отработал:", err)

    if points:
        status = "конечное число решений"
```

```
    elif family:
        status = "параметрическое семейство"
    elif used:
        status = "решений нет"
    else:
        status = "решить не удалось"
    return {"points": points, "family": family, "status": status, "methods": used}


def jacobian():
    return sp.Matrix([[sp.diff(c, v) for v in variables] for c in constraints])


def singular_system():
    """h_j = 0 и равенство нулю всех миноров Якоби порядка m (там ранг < m)."""
    J = jacobian()
    m, n = J.shape
    eqs = list(constraints)
    if m <= n:
        for cols in combinations(range(n), m):
            eqs.append(sp.expand(J[:, list(cols)].det()))
    return eqs


def classify(L, values):
    """Второй порядок: гессиан L сужаем на касательное подпространство."""
    J = jacobian().subs(values)
    basis = J.nullspace()
    if not basis:
        return "касательное пространство нулевое, второй порядок не применим"
    B = sp.Matrix.hstack(*basis)
    H = sp.hessian(L, variables).subs(values)
    Hr = sp.simplify(B.T * H * B)

    eig = []
    for val, mult in Hr.eigenvals().items():
        num = complex(sp.N(val))
        if abs(num.imag) > EPS:
            return "комплексные собственные числа, классификация не удалась"
        eig += [num.real] * mult

    if all(e > EPS for e in eig):
        return "строгий локальный минимум"
    if all(e < -EPS for e in eig):
        return "строгий локальный максимум"
    if any(e > EPS for e in eig) and any(e < -EPS for e in eig):
        return "не экстремум (седло)"
    return "второй порядок ответа не дает"


def show_point(d, names):
    return "(" + ", ".join(f"{n} = {d[n]}" for n in names) + ")"


# ---------- основная программа ----------

def main():
    print("Целевая функция: f =", f)
    print("Ограничения:")
    for j, c in enumerate(constraints, 1):
        print(f"  h{j} = {c} = 0")
```

```
# множители и функция Лагранжа
lams = list(sp.symbols(f"lam1:{len(constraints) + 1}", real=True))
L = sp.expand(f + sum(l * c for l, c in zip(lams, constraints)))
print("\nФункция Лагранжа:\n  L =", L)

# система стационарности + исходные ограничения
stationarity = [sp.diff(L, v) for v in variables]
system = stationarity + constraints
unknowns = variables + lams

print("\nУравнения стационарности:")
for v, e in zip(variables, stationarity):
    print(f"  dL/d{v} = {e} = 0")
print("Ограничения:")
for j, c in enumerate(constraints, 1):
    print(f"  h{j} = {c} = 0")

print("\nМатрица Якоби ограничений:", list(jacobian()))

# решение системы Лагранжа
res = solve_all(system, unknowns)
print("\nРешение системы Лагранжа:")
print("  решатели:", ", ".join(res["methods"]))
print("  статус:", res["status"])
print("  найдено действительных решений:", len(res["points"]))

# особые точки
sing = solve_all(singular_system(), variables)
print("\nПоиск особых точек (rank J < m):")
print("  статус:", sing["status"])
print("  найдено точек:", len(sing["points"]))

# собираем кандидатов
cands = []
for sol in res["points"]:
    pt = {v: sol[v] for v in variables}
    if not all(is_zero(c.subs(pt)) for c in constraints):
        print("  отброшено, ограничение не выполнено:", show_point(pt, variables))
        continue
    cands.append({"pt": pt, "lam": {l: sol[l] for l in lams}, "kind": "Лагранж"})
for sol in sing["points"]:
    pt = {v: sol[v] for v in variables}
    if any(same_point(pt, c["pt"], variables) for c in cands):
        continue
    cands.append({"pt": pt, "lam": None, "kind": "особая"})

# анализ каждого кандидата
J = jacobian()
m = len(constraints)
print("\nКандидаты:")
for i, c in enumerate(cands, 1):
    pt = c["pt"]
    c["value"] = sp.simplify(f.subs(pt))
    c["resid"] = [sp.simplify(h.subs(pt)) for h in constraints]
    c["feasible"] = all(is_zero(r) for r in c["resid"])
    c["rank"] = int(J.subs(pt).rank())
    if c["lam"] is not None:
        c["class"] = classify(L, {**pt, **c["lam"]})
    else:
        c["class"] = "особая точка, метод Лагранжа неприменим"
```

```
        print(f"\nКандидат {i} ({c['kind']})")
        print("  точка:", show_point(pt, variables))
        if c["lam"] is not None:
            print("  множители:", show_point(c["lam"], lams))
        print("  f =", c["value"])
        print("  невязки:", c["resid"])
        print("  допустимость:", "да" if c["feasible"] else "нет")
        print(f"  ранг Якоби: {c['rank']} из {m}")
        print("  классификация:", c["class"])

    # итог
    print("\n" + "=" * 50)
    print("Итог")
    print("=" * 50)

    can_declare = (
        SET_IS_COMPACT and CANDIDATES_COMPLETE and cands
        and res["points"] and not res["family"]
        and sing["status"] in ("решений нет", "конечное число решений")
        and not sing["family"]
        and all(c["feasible"] for c in cands)
    )
    if not can_declare:
        print("Глобальные экстремумы автоматически не объявлены:")
        print("не выполнено хотя бы одно из условий (компактность, полнота")
        print("списка, конечность решений, допустимость). Нужен ручной анализ.")
        return

    best = min(cands, key=lambda c: float(sp.N(c["value"])))
    worst = max(cands, key=lambda c: float(sp.N(c["value"])))
    print("Глобальный минимум:", show_point(best["pt"], variables), "f =", best["value"])
    print("Глобальный максимум:", show_point(worst["pt"], variables), "f =", worst["value"])
    print("Основание: допустимое множество компактно, f непрерывна,")
    print("кандидаты перечислены полностью, значения f сравнены.")


if __name__ == "__main__":
    main()
```